Document number:	P2388R1
Date:	2021-08-14
Audience:	SG21
Reply-to:	*Andrzej Krzemieński <akrzemi1 at gmail dot com>* *Gašper Ažman <gasper dot azman at gmail dot com>*

Minimum Contract Support: either *Ignore* or *Check_and_abort*

This paper proposes a minimal contract support framework for C++. It is the first phase of the plan in [P2182R1]. As such, it is the Minimum Viable Product (MVP) achieving the following goals:

1. Be a coherent whole, and provide a value to multiple groups of developers.
2. Be small enough to progress through the standardization pipeline for c++23.
3. Side-step all controversial design issues to obtain consensus.

As a sketch: a function containing an assertion annotation:

```
bool fun(int a, int b)
{
  [[assert: are_compatible(a, b)]]; // assertion annotation
  return transform(a, b);
}
```

In *Check_and_abort* mode, this renders runtime code equivalent to:

```
bool fun(int a, int b)
{
  [&]() noexcept {
    if (are_compatible(a, b)) {} else {
      CONTRACT_VIOLATION_HANDLER("are_compatible(a, b)");
      std::abort();
    }
  }(); // immediately invoked lambda

  return transform(a, b);
}
```

That is: a runtime check is performed, and if it returns something `false`y, an implementation-defined message is displayed to the stream buffer associated with the object `stderr` and `std::abort()` is called. If any exception is thrown during the evaluation of the predicate or the logging statement, `std::terminate()` is called.

In *Ignore* mode, the code is rendered without calling the lambda (thus odr-using the predicate, but not evaluating it).

0. Revision history {rev}

0.1. R0 → R1{rev.r01}

1. Added the proposed wording.
2. Added new design point: incompatible contract annotations in different translation units.
3. Added new design point: referencing value and rvalue references function arguments in postconditions.
4. Resolved open issue: objects are not treated as const in contract predicates.
5. Resolved open issue: implementations may (but don't have to) runtime-check contract annotations in core constant expressions even in *Ignore* mode.
6. Added an open issue: alternate postcondition syntax.
7. Expanded the design details and rationale section, based on feedback from SG21.

1. Terminology {term}

In this document we use the following terms recommended in [P2038R0] and [P2358R0]:

Abstract machine violation
: This is what the Standard defines as Undefined Behavior in Clause 4 through Clause 15, which is often called a "hard undefined behavior" or "language undefined behavior", which includes things like `++INT_MAX`, `1/0` or null pointer dereference.

BizBug
: A situation where a function is invoked in a way that is disallowed by its specification; or when it returns a value or has a side effect disallowed by the specification. (This presupposes the existence of function "specification".)

Contract annotation
: Declaration of a precondition or a postcondition or an assertion, such as `[[pre: i != 0]]` or `[[assert: x != y]]`. Contract annotations can express a subset of function specification.

2. Proposal {ovr}

There are three kinds of contract annotations: **preconditions** and **postconditions** on function declarations, and **assertions** inside function bodies.

Contract annotations use an attribute-like syntax:

```
int select(int i, int j)   // first declaration
  [[pre: i >= 0]]
  [[pre: j >= 0]]
  [[post r: r >= 0]];      // r names the return value

int select(int i, int j);  // subsequent declarations can
                           // repeat or ignore the annotations

int select(int i, int j)   // the definition
{
  [[assert: _state >= 0]];

  if (_state == 0) return i;
  else             return j;
}
```

We specirfy the semantics of two modes of program translation:

1. ***Ignore***: compiler checks the validity of expressions in contract annotations, but the annotations have no effect on the generated binary. Functions appearing in the predicate are ODR-used.
2. ***Check_and_abort***: each contract annotation is checked at runtime. The check evaluates the corresponding predicate; if the result equals `false`, an implementation-defined *contract violation handler* is invoked.

We recommend (the ISO word "should") that the default mode is *Check_and_abort*.

Check_and_abort contract violation handler semantics

The semantics of the contract violation handler are implementation-defined, subject to the following constraint: **control never leaves the violation handler**. It can stop the program, hit a breakpoint, or run forever after emailing a sysadmin, for instance. If the program exits, it must signal unsuccessful termination appropriately for the context, such at returning a non-zero error code (as with `abort()`). We recommend (the ISO word "should") that the violation handler output a diagnostic message to the standard diagnostic stream and call `std::abort()`. This corresponds to what [P2358R0] calls "Unspecified but never returns" and what [P1606R0] calls "check_never_continue".

Name lookup in pre-/post-conditions

Name lookup for preconditions and postconditions is performed as in the trailing return type.

Access control

Access control is performed as if the predicate was evaluated inside the body of the function, making private and protected members of the containing class accessible to predicates of preconditions and postconditions of member and befriended functions.

Preconditions and postconditions are checked (in *Check_and_abort* mode) even if the function is called through a pointer.

Side-effects

Side effects are supported in the predicates of contract annotations. Implementations are allowed to both remove or duplicate these side effects, so they cannot be relied upon. This is similar to the existing rules around eliding copy-constructor side-effects.

Looking at the source code with contract annotations alone — that is, not looking at compiler switches — the programmer has no guarantee if predicates in contract annotation will be evaluated at runtime or not, or if the program is stopped after such evaluation. Whether the side effects of predicates are evaluated (one or more times) or not — even in *Check_and_abort* mode — is unspecified.

Mixed-mode linking

Implementations are allowed to provide a mixed mode, where some translation units are translated in *Ignore* mode and others in *Check_and_abort* mode. This may be necessary for linking the user program with compiled libraries.

3. Deferred features

These features are deferred due to unresolved issues. This proposal makes sure not to block any of these being proposed in the future.

- assertion levels: unresolved issues around fallthrough, among many other things. Expecting future proposals in this space.
- continuation mode: unresolved issues around fallthrough. Expecting future proposals in this space.
- ability to install a custom violation handler: unresolved issues around pretty much everything.
- a way to express preconditions and postconditions for lambdas: name lookup is already problematic in lambdas in the face of lambda captures. This problem is pursued in [P2036R1], and until it has been solved we see no point in delaying the minimum contract support proposal.

4. FAQ: can I do X?{faq}

4.1 What you can do with this proposal:

1. Programmer: The ability to declare parts of your functions' contract in the source code in a portable way. This alone adds value: it may help you better understand your function contracts. It may help your users understand your functions' contracts.
2. Programmer: The ability to run your program locally in mode that checks all your assertions expressed in contract annotations, and aborts the program if any bug is detected. (This experience is similar to using C `assert()`, except that you can put annotations in more places.)
3. Programmer: The ability to build your program in a mode that adds not a single instruction to your code, even if you put contract annotations.
4. Static analyzer vendor: contract annotations written by your users can help you generate new and better diagnostics.

4.2 What you cannot do with this proposal:

1. Programmer: You may not be able to (depending on the compiler) to runtime-check contract annotations in your code but ignore them in the library you link with. Libraries may need to ship binaries in two versions: with contract checking enabled and disabled. (The Standard Library does not have to introduce contracts.)
2. Programmer: You will not be able to selectively disable the checking of some contracts. If some checks would be too expensive, you would have to leave them out unannotated.
3. Programmer: You will not be able to run the program in test mode, where contract violations are reported but program is allowed to execute further.
4. Library vendor: You will not be able to insert "experimental" contract annotations without the risk of terminating your user's program.
5. Library vendor: throwing from a failed contract test is not a supported way of unit-testing contract annotations. Death-tests are available in popular testing frameworks such as google test and should work for this scenario.

We consider these use cases valid and important. Our choice is not to support them in the MVP, but to ultimately enable their support in the future. Their omission from this paper serves one purpose: enable programmers to declare contract annotations as soon as possible.

4.3. How does linking with 3rd-party libraries work? {ovr.lib}

A library does not have to declare contract annotations (even the Standard Library). If so, this is as now.

Libraries that declare contract annotations and ship compiled binaries will have to ship two versions: one compiled in *Ignore* mode, and one in *Check_and_abort*. This also applies to the Standard Library.

One can also see toolchains supporting mangling the contract mode into the symbol name and emitting code for both into the same fat binary.

5. Motivation {mot}

The motivation for adding contract support to the language has already been provided multiple times, for example in [N1613], [N1800], [N4110], [N4075], [N4135], [P0380R0], [P2182R1]. In brief, contract annotations allow the programmers to encode what they consider a bug in code. This information can be used by tools in a number of ways.

The motivation for starting with a minimal set of features has been provided in [P2182R1]. In summary, it is to minimize the risk of having the discussion or disagreement about the secondary features of the contract support framework impede or prevent the addition of the primary functionality: the ability for the programmer to communicate what is considered a bug in the program. The plan for the MVP is therefore to find a minimal uncontroversial subset, gather consensus around it, polish, and deliver it quickly.

The secondary features can then be proposed on top of this stable core; but, since it has shipped, the core part of contract support framework will not be at risk of being removed or postponed even if they run into issues.

We have observed that even the portions of [P2182R1] arose controversies. This paper to removes these portions.

5.1. Two programming models {mot.two}

There are at least two views on what a contract violation means among the stakeholders.

One view is that contract violation indicates a bug, and this situation is as serious as dereferencing a null pointer or making an invalid access to memory. If this situation is detected at run-time, the only acceptable default action is to immediately abort the execution of the program to prevent any further damage.

The other view is that the contract violation is just a piece of information, usually about a bug (but not always: for instance, in unit tests a programmer may violate a contract on purpose), that can be processed in a number of ways. Program termination is one, but the program might as well just continue with the normal execution or jump to a different location, for example, by throwing an exception.

The two views conflict in whether anything other than program termination should be allowed after a runtime-confirmed contract violation, but they also have overlapping parts:

1. The common syntax for contract annotations.
2. The common understanding that contract violation, at least outside of unit tests, indicates a bug, which programmers intend to avoid or fix.
3. Even the second view allows for the mode where the program is just aborted.

We believe that this overlapping part is already useful for many groups of programmers. It allows the programmers to declare what they know to be a bug in their components. It allows tools other than compilers — such as static analyzers or IDEs — to diagnose or help diagnose bugs in programs statically. The *Check_and_abort* mode gives a guarantee to programmers that if a bug is detected at run-time, the program — apart from aborting — will do no further damage. This enables UB-sanitizers to diagnose not only abstract machine violations ("hard" UB) but also contract violations in a uniform way (provided that this way does not require the program to continue after the reported violation).

Because this proposal is — we believe — a common part of the two views, we hope that proponents of either view should find nothing unacceptable, other than that it is missing features.

5.2. Is the proposal not too small? {mot.too}

It may appear that what is proposed is too small and not usable in many real-life use cases. In particular, the ability to continue the program execution after a runtime correctness check has failed is brought up in this context.

In section {rat.con}, we describe why the design of the continuation mode, such that it avoids introducing surprising new reasons for undefined behavior and gathers consensus, will take a long time, and thus not be ready for C++23.

We can see two options:

1. Deliver a feature-rich solution for C++26, or later, or never.
2. Deliver a reduced set of features for C++23; deliver the remaining features for C++26, or later or never.

The latter option subsumes the former, even if one disagrees that the reduced feature set is "viable" for one's use cases.

6. Design details {des}

6.1. Design goals {des.gol}

We want to provide a uniform notation, usable in different contexts — in this proposal these contexts are function declarations and function bodies — for expressing runtime conditions, the violation of which necessarily constitutes a bug in the program. Such information can be consumed in a number of ways, by many different tools, most of which are outside of the scope of the C++ Standard; we want all the tools to consume the same notation.

We want the proposal — the number of features, the amount of wording changes — to be as small as possible, and remain as open for future extensions as possible. The goal is to deliver a small piece fast. We demonstrate how additional — often requested — features can be added on top of this proposal while preserving backwards compatibility.

We want the proposed to be zero-overhead: programmers should be able to add contract annotations and compile the program in a mode that doesn't add a single instruction to the compiled program.

We want other programmers to get the guarantee that the program never continues once it has been determined, through the correctness annotations, that it has encountered an illegal state. This goal seems somewhat contradictory with the previous one, but we manage to combine them through the introduction of *translation modes*. This means that a correctness annotation can have different effects on the compiled program, depending on what translation mode it was compiled with.

6.2. Predicates {des.pre}

In addition to the grammar of contract annotations, this proposal includes the facility that uses them: the ability to check them at runtime, and stop the program if the check fails.

```
int fun(int i)
{
  [[assert: i != 0]]; // correctness annotation
  return i;
}

int main()
{
  fun(0);
  std::cout << "hello\n";
}
```

Running this program can have two different effects, depending on whether we enable the generation of runtime-checks. If we don't, the program will output "hello" and exit 0; but if we enable them, this program will exit via `std::abort()` before printing "hello".

`[[assert:` *Pred*`]]` is a correctness annotation: one that appears inside function body, as a statement. *Pred* names a predicate. If the predicate returns `false` it means we have a bug. If it returns `true`, we do not know if we have any bug or not.

In order for such a runtime test to work, *Pred* has to be a valid C++ expression convertible to `bool`. However, it differs slightly from just any other expression.

Another consumer of contract annotations is a static analyzer that is able to perform a symbolic evaluation of the program and determine if any control path leads to a bug. (Correctness annotations provide a formal definition of a bug.)

```
int get_count();

int fun(int i)
  [[pre: i > 0]] // correctness annotation
{
  return i - 1;
}

int main()
{
  int count = get_count();
```

```
  if (i < 0)
    return fun(-i);
  else
    return fun(i);
}
```

`[[pre:` *Pred*`]]` is *precondition*: it is another correctness annotation: one that appears in a function declaration. It says that whenever this function is called — just after the function arguments have been initialized — the predicate must *hold*, otherwise we have a bug.

The static analyzer could determine that if function `get_count()` returns 0, then function `fun()` is called such that the predicate in the precondition does not hold.

Here, a precondition "hold"ing implies having a predicate in the mathematical sense: one that returns a true-false answer, and has no effect on the environment, since in mathematics there is no such thing as an "environment". However, in an imperative language like C++, expressions have *side effects*. The expression in the precondition above doesn't have side effects, so the meaning is clear, but a static analyzer would have a hard time (and we also) if the precondition on the function read:

```
int fun(int i)
  [[pre: --i >= 0]]
{
  return i;
}
```

Therefore, while the predicates in contract annotations are C++ expressions, we take some measures to address the situations where these expressions have side effects. Thus, they are not 100% C++ expressions.

Our choice of notation accounts for the fact that we have to address these two, quite different, use cases.

Features added in the future may further increase the difference between predicates in contract annotations and normal C++ expressions. One such extension would be a notation for inexpressible conditions as discussed in [P2176r0].

6.3. Naming the return value {des.ret}

A postcondition expresses the effects that a function guarantees when:

- its preconditions are satisfied, and
- when it returns normally (not via an exception or `longjmp`).

One of such guarantees is the state of the returned value:

```
char get_digit()
  [[post c: is_digit(c)]];
```

`[[post` *Name*`:` *Pred*`]]` is a *postcondition*: it is another correctness annotation that appears in a function declaration.

In a function declaration the return value (or reference) doesn't have a name, so there is a way to introduce it. The predicate can refer to this name. The type of the return value is usually visible in the declaration, except when we use a return placeholder type and do not provide the definition from which the type could be deduced:

```
auto get_digit()
  [[post c: is_digit(c)]]; // error: decltype(c) unknown
```

The compiler has to parse the contract annotation in order to determine if the expression is well formed, but this task is impossible when we do not know the type of the object. This problem does not occur in a function template declaration, because the task of verifying the validity of the expression is deferred until instantiation time:

```
template <int I>
auto get_digit()
  [[post c: is_digit(c)]]; // OK
```

The problem also does not occur when we provide the body of the function from which the type of the return value can be deduced:

```
auto get_digit()
  [[post c: is_digit(c)]] // OK: decltype(c) is char
{
  return '0';
}
```

This has been discussed in detail in [P1323R2].

6.4. Function arguments in postconditions {des.arg}

A postcondition may describe a relation between the returned value and and function arguments:

```
int generate(int lo, int hi)
  [[pre lo <= hi]]
```

```
  [[post r: lo <= r && r <= hi]];
```

The intention of the above declaration seems quite obvious. The function guarantees that:

- The objects the user used to call the function will not be modified.
- The generated number will fall between the limits that the user passed.

The postcondition needs to have access to the returned value, so it is called after the function has initialized it. In order for the postcondition to check the condition that the caller expects, we have to make sure that the function has not modified the values of function arguments. In order to achieve that we require that non-reference function arguments referenced in the postcondition must be declared const in function definition:

```
int generate(int lo, int hi) // error: hi and lo not const
{
  return lo;
}

int generate(const int lo, const int hi) // OK
{
  return lo;
}
```

This may have the effect that if the function argument is returned from the function, the added `const` will change the implicit move into a copy, which may add an overhead, and may make the program ill-formed, if the type is not copyable. However, this will not happen behind the programmer's back. the programmer is first informed (in form of an error message) that the argument has to be declared `const`. It will be the programmer's decision to make this change. An alternative decision might be to not declare postcondition annotations on functions that cannot be rewritten so that they take their postcondition-referenced arguments as `const`.

Another way to put this is that postconditions referring to non-reference function arguments make these arguments part of the interface: the function no longer has freedom to use it at will.

This restriction does not apply to references:

```
int generate(int& lo, int& hi) // OK
  [[pre lo <= hi]]
  [[post r: lo <= r && r <= hi]]
{
  return lo;
}
```

Pointers are values, so when they are passed by value, they are subject to the same constraints as other types passed by value.

6.5. Side effects in predicates {des.eff}

Side effects are allowed in predicates. Some side effects are desired, as described in section {rat.eff}, but they have to be "self contained": removing them (or a well defined portion thereof) should not affect the program behavior.

Generally, a compiler is allowed to apply code transformations as if the expressions in contract predicates were side-effect-free. We do not require or expect that the predicates are referentially transparent, because it is often desired to refer from them to global variables, even if they are mutable.

The compiler is allowed to replace any subexpression appearing in a contract annotation with just the value of this expression. There are two cases where the compiler has enough information to do this:

1. When it can peek inside the expression and observe that in order to provide the result, it doesn't have to evaluate the entire function body:

   ```
   bool is_positive(int i)
   {
     printf("is_positive() called\n"); // this does not affect the result
     return i > 0;
   }
   ```

2. When the same expression has already been evaluated during the same sequence of precondition and postcondition tests:

   ```
   int produce() [[post r: p(r) && q(r)]];
   void consume(int i) [[pre: p(i)]] [[pre: q(i)]];

   consume(produce());
   ```

   Here, it is enough to call `p()` and `q()` only once, and compiler is allowed to reuse these results rather than calling `p()` and `q()` again.

However, for the second case to work, an evaluation of any predicate in the sequence cannot have side effects that would affect the results of other predicates in the sequence. This cannot be enforced statically, so we call it undefined behavior.

It is undefined behavior if evaluation of one predicate or subexpression thereof affects the result of any other predicate or a subexpression thereof in the same test sequence. It is also undefined behavior if the evaluation of a predicate modifies the value of function arguments, the returned value, or any other objects odr-used by the predicate. Any other side effects are allowed and valid.

```
void f(int i) [[pre: ++i]]; // UB if evaluated
```

The compiler is also allowed to call the predicate twice:

```
void f(int i) [[pre: p(i)]] [[pre: q(i)]];
```

When function `f()` is evaluated, it might cause the following sequence of precondition tests: `p(i)`, `q(i)`, `p(i)`, `q(i)`. This allows the implementations to test the preconditions both inside and outside the function body. This ability is useful for generating good error messages from preconditions: they have to be evaluated inside the function, in case the call is made through a pointer. But when a call is made directly, we can evaluate it outside, and then the error message points at the caller, rather than the callee.

A possible, hypothetical but valid, implementation of a contract annotation check would be to start a transaction, evaluate the predicate, save the result on the side, cancel the transaction (thereby erasing all side effects since the beginning of the transaction), and just using the stored value instead as a result.

6.6. Indirect function calls {des.ptr}

Function pointers and references cannot have contract annotations, but functions with contract annotations can be assigned to them:

```
using fpa = int(*)(int) [[pre: true]]; // error

using fptr = int(*)(int);
int (int i) [[pre: i >= 0]];

fptr fp = f;  // OK
fp(1);        // precondition is checked
```

In other words, contract annotations are not part of function type. Thanks to this decision, you can write code like:

```
int fast(int i) [[pre: i > 0]];
int slow(int i) [[pre: true]];  // no precondition

int f(int i)
{
  int (*fp) (int) = i > 0 ? fast : slow;
  return fp(i);  // if fast() is called, its precondition is checked
}
```

The consequence of this behavior is that an implementation cannot check the precondition in the place where function is called. The check has to be performed inside the function. Therefore, it is a reasonable implementation strategy to implement the precondition check inside the function in case it is called indirectly, but where the function is called directly, check it in the caller, in order to provide the source location that is the culprit of contract violation. This may sometimes result in checking the same precondition twice.

The same mechanism works for function wrappers:

```
using fp = int(*)(int);
int f(int i) [[pre: i >= 0]];

function<int(int)> fp = f;  // OK
fp(1);   // precondition is checked
```

6.7. Virtual functions {des.vir}

You can put contract annotations on virtual functions, however they behave in a certain way in the overriding functions. You can omit contract annotations in the overriding function. The overriding function then has the same contract annotations as the virtual function in the base class, and the names in the predicates are looked up in the context of the base class.

```
static const int N = 1; // #1

struct Base
{
  virtual void f() [[pre: N == 1]];
};

template <int N>       // N is shadowed
struct Deriv : Base
{
  void f() override;
};

int main()
{
  Deriv<2>{}.f(); // precondition test passes
}
```

The precondition in the overriding function is `N == 1`, but the name lookup is performed in the context of class `Base`, so it sees the global variable `N` declared in line #1.

You cannot declare a contract annotation in the overriding function if the virtual function in the base class doesn't have a corresponding contract annotation.

You can declare a contract annotation in the overriding function, but it has to be identical (modulo the names of function parameters) to the corresponding contract in the overridden function. However, the program is ill formed, no diagnostic required, if name lookup in the predicate finds different entities than if the name lookup were performed in the context of the base class:

```
static const int N = 1; // #1

struct Base
{
  virtual void f()
    [[pre: N == 1]]
    [[post: sizeof(*this) < 100]];
};

template <int N>       // N is shadowed
struct Deriv : Base
{
  int i[100];

  void f() override
    [[pre: N == 1]]                 // IF-NDR, finds different N
    [[post: sizeof(*this) < 100]];  // IF-NDR, inspects different class
};
```

We do not allow the preconditions in the overriding function to be "wider" and the postconditions to be "narrower" than in the overridden function. Inheritance also means inheriting the contract. If you need the overridden function to be called out of contract of the base class, you have to provide a separate function:

```
struct Base
{
  virtual int fun(int i)
    [[pre: i >= 0]]
    [[post r: r >= 0]];
};

template <int N>
struct Deriv : Base
{
  int fun_wide(int i)
  {
    return i;
  }

  virtual int fun(int i)
    [[pre: i >= 0]]
    [[post r: r >= 0]]
  {
    return fun(int i);
  }
};
```

The above example also illustrates that a feature request "I want the postcondition in the overriding to be the same or narrower than the postcondition in the overridden function" is conceptually wrong. Function `fun_wide()` has a wider postcondition than `fun()`, but it is still substitutable for `fun()`. This is because `fun_wide()` will never return a negative number as long as it does not obtain a negative number as input. A postcondition can never be treated in isolation from the function's precondition.

7. Design rationale {rat}

7.1. Why not use attributes {rat.att}

An alternative syntax for contract annotations would be to use attributes:

```
// not proposed
int f(int i)
  [[pre(i >= 0)]]
  [[post(r: r >= 0)]];
```

We do not propose this for two reasons. One is social: we do not want to depart from what the previous proposal settled upon and what EWG agreed to. Second is technical. This syntax will not work for assertions in function bodies:

```
int f(int i)
{
  [[assert(i >= 0)]]; // interpreted as macro
  return i;
}
```

Here, the `assert` is interpreted as C macro and expanded, causing a compiler error. This problem is not present when identifier `assert` is followed by a colon.

This problem would disappear if we used some other name than `assert`. However, name `assert` is a well established name, not only in C++, and abandoning it would make the learning curve unnecessarily steeper.

7.2. Incompatible contract annotation across translation units

Because we do not require contract annotations to be repeated on subsequent function declarations, it is possible that two translation units can see different sets of contract annotations for the same function:

```
// translation unit 1              // translation unit 2
// ------------------              // ------------------

// first declaration in this TU:   // first declaration in this TU:
void f(int i) [[pre: i > 0]];      void f(int i); // no precondition

void f(int i)                      int main()
{                                  {
  // ...                             f(0); // precondition checked or not?
}                                  }
```

In this context, the question whether pre- and postconditions "belong" to the caller or the called function, obtains a new dimension. A similar issue occurs for default function arguments. Ideally, from the user perspective, we would like the above situation to be an error diagnosable by the implementations. However, because this cannot be diagnosed from a single translation unit, it may be too burdensome to require of the implementations to diagnose it.

This looks similar to ODR violation. The consequences in the run-time behavior in *Check_and_abort* mode are that the wrong set of contract annotations gets executed or that both sets are executed, if the implementation decides to test the pre-/postconditions both inside and outside the function.

However, inconsistent contract annotations have consequences on static analysis. What conditions should a static analyzer look for? It may turn out to be incompatible with the runtime checks.

We can see a couple of ways to address this.

1. If such incompatible declarations occur in the program, it is ill-formed, no diagnostic required (IF-NDR).
2. It is ok to have such incompatible declarations as long as the function is not called. If it is, the behavior is undefined (UB).
3. Allow calling such function, but make it unspecified which set of pre-/postconditions, or both, is called in Abort_and_continue mode.

We choose option 2 (undefined behavior). It makes more programs well formed than option 1. We do not go with option 3 in order to keep consistency with static analysis: if the program is correct (no UB, no IF-NDR) runtime checks are compatible with static analysis results.

While option 3 opens no new undefined behavior (either UB or IF-NDR), it would make the runtime checks behave in an incompatible way with static analysis. We have to know what set of contract annotations a function has. The answer cannot be "it depends on which file you are compiling".

For a similar reason we do not go with option 2, even though option 1 leaves fewer situations undefined. Simply having a function with two different sets of contract annotations is an error, even if this error cannot be diagnosed by the implementation.

7.3. Why require `const` for non-reference args? {rat.arg}

As indicated in section {des.arg}, a non-reference function parameter referenced in a postcondition must be declared `const` in function definition. This is to prevent any modification of the parameter.

In order to show why this is necessary, consider the following example:

```
// declaration that the user sees:
int generate(int lo, int hi)
  [[pre lo <= hi]]
  [[post r: lo <= r && r <= hi]];

// definition that only the author sees:
int generate(int lo, int hi)
{
  int result = lo;
  while (++lo <= hi) // note: lo modified
  {
    if (further())
      ++result;      // incremented slower than lo
  }
  return result;
}
```

Because `lo` is modified as the function is executed, we get an ambiguous situation where:

- the returned value is no smaller than the *initial* value of `lo`;

- but it might be smaller than the value of `lo` as observed upon the exit from the function.

We could just followed the rule: postcondition is evaluated at the end of the function, and whatever values function arguments have at that point, we read them. This would make the model for postconditions "simple", but it would give the semantic meaning to the postcondition that is clearly against the intent of the contract:

```
int generate(int lo, int hi)
  [[pre lo <= hi]]
  [[post r: lo <= r && r <= hi]];
```

The consumers of the contract only see the declaration. And the declaration clearly states:

- The objects you passed to my functions will not be modified (because I will make copies).
- The generated number will fall between the limits that you passed (not between the values that I created during the course of function execution).

This is how humans will read this declaration; and this is how the tools (like static analyzers) will read this declaration. So, in order for both tools and the C++ runtime checks to have the same meaning, we have to exclude some otherwise valid implementations of the function. The easiest way is to require that these parameters be visibly `const`.

A similar problem, albeit more difficult to spot, is when the function argument is implicitly moved from:

```
// declaration:

string forward(string str)
  [[post r: r == str]];

// definition:

string forward(string str) // disallowed in our proposal
{
  // ...
  return str; // implicit move
}             // postcondition reads the moved-from state
```

If we don't require the argument to be `const` we end up with the postcondition reading the moved-from state. A programmer may not be even aware that a move is taking place. Adding `const` prevents the move.

Why do we require it only for non-reference arguments?

First, there is no way to require it for reference arguments: they cannot be `const`. Second, there is no need to. Reference function arguments clearly indicate in the declaration that we are interested in modifying the objects that the caller sees. In fact this may be the desire of the caller to have the called function modify these objects, and this can even be reflected in the postcondition:

```
void fix_limits(int& lo, int& hi)
  [[post: lo <= hi]];
```

Is this measure enough? What about pointers? Even if they are const, the value they point to can be mutated.

The answer is the same as for the references: you can clearly see from the declaration that the function is allowed access to objects from your scope, so you are prepared and maybe expect them to change:

```
void fix_limits(int* lo, int* hi)
  [[post: *lo <= *hi]];
```

It has to be kept in mind that pointers, including smart pointers, are value-semantic types: their value is the address: not the value they point to.

The protective measure with adding `const` does not work for types that violate value semantics:

```
class Book
{
  shared_ptr<string> _title = make_shared<string>(); // poor choice

public:
  string const& title() const { return *_title; }
  void set_title(string t) { *_title = std::move(t); }
  friend auto operator <=>(Book const&, Book const&) = default;
};
```

When we copy objects of this type, the copy is tied to the original, so when we modify the copy, this automatically modifies the original. We believe it is a fair compromise to not take care about types like this.

The downside of the solution with adding `const` is that when you add a postcondition to a function declaration, it can cause the definition of the function to be come ill-formed. Adding `const` may not suffice, because the implementation might be modifying function parameters.

There are some combinations of types and functions for which an alternative implementation may not exist if we make function parameters `const`. Consider the following example by Ville Voutilainen:

```
struct X { int v = 0; };

void f(propagate_const<shared_ptr<X>> x)
  // [[post: x->v == 42]]
{
  x->v = 42;
}
```

For such cases, programmers will not be able to declare a postcondition.

Other options for solving non-reference arguments in postconditions have been considered. C++20 allowed them, but called it undefined behavior if such parameter was modified during the evaluation of the function. This included the situation where the parameter was implicitly moved from.

That solution encouraged too many silent bugs. The present solution looks superior. First, potential bugs are detected at compile time. Second, we leave more doors open for the future. An illegal declaration can be turned into a legal declaration with undefined behavior. The reverse is not possible.

Another option would be to, rather than requiring the programmer to manually type `const` on function arguments, make these arguments implicitly const. This spares some typing, but has downsides. It is not as future proof as the proposed solution: the syntax is well formed with well defined semantics, so we cannot change it in the future. Second, this alternative can cause confusion. When an argument is changed implicitly to `const`, overload resolution starts selecting different overloads, and the programmer cannot easily figure out why, as there is no `const` in sight.

Another option: disallow postconditions to reference function arguments. This is possible, but the proposed solution is superior. Referencing non-const non-reference parameter is already an error; we just make it easier for people who can afford to put `const` in the definition.

Another option: silently make a copy of every non-reference parameter referenced in the postcondition. this copy would be only visible to the postcondition, so there is no way a function can possibly change it. While attractive, this would require making silent copies. This may not be satisfactory if a type is expensive to copy, or simply non-copyable, and is a bad trade-off in situations where the function does not modify the parameter anyway. It is a possible future opt-in extension, described in section {fut.old}.

Finally, there is an option to remove postconditions from this proposal. This will however make the same problem reappear once we try to add postconditions in the future.

7.4. Why disallow continuation (for now) {rat.con}

The behavior of runtime-checking, logging but not aborting has a number of surprising and non-obvious effects.

First, in case the precondition is guarding against the Abstract Machine Violation (hard UB), reaching the point of the abstract machine violation might result in what would be observed as removing the log entry for the previous contract violation. This is discussed in detail in appendix A.

Second, a mechanical transformation of every contract annotation to "check, log and continue" can introduce a new Abstract Machine Violation (hard UB) if the programmer assumes the short-circuiting behavior for subsequent contract annotations. Consider the following example.

```
int f(int * p)
  [[pre: p]] [[pre: *p > 0]]
{
  if (!p)      throw Bug{};  // safety double-check
  if (*p <= 0) throw Bug{};  // safety double-check

  return *p - 1;             // (*) business logic
}
```

This program upon `p == nullptr` behaves in a way that (1) does not cause abstract machine violation and (2) guarantees that the business logic, indicated with *, is never reached. This happens for both *Ignore* and *Check_and_abort* mode. However, in a mode that allows the program to continue, this causes abstract machine violation upon runtime-checking the second precondition. This is explained in detail in appendix A.

We believe that "the continuation mode" is a useful feature that is necessary for some essential applications (like adding contracts in libraries in stages). Our motivation for omitting it from the MVP is the timing concerns: we want to deliver a small but relatively useful feature quickly.

7.5. Why disallow throwing on contract violation (for now) {rat.exc}

We propose to disallow throwing upon detected contract violation because it falls outside of one of the presented programming models: the one that says that it is unacceptable to allow the program with a detected bug to continue.

By disallowing throwing we also avoid exploring and describing many aspects observable behavior that this would trigger:

1. How contracts interact with `noexcept`.
2. If the precondition is evaluated before the function call or inside the function.

This makes the proposal smaller, and therefore more likely to progress faster through the WG21 process.

7.6. Why no user-provided violation handler (for now) {rat.hnd}

Not proposing the ability to install user-provided violation handlers is again motivated by timing constraints. This way we avoid the necessity to specify the interface for the violation handler and its constraints.

However, the way we specify the handler (mostly implementation defined) allows for things like programmer installing a callback in an implementation-defined way.

7.7. Why allow side effects in predicates {rat.eff}

We intuitively expect that predicates in contract annotations have no side effects. This expectation is reflected in terms like "if the precondition *holds*". However, it is impossible to enforce such constraint in an imperative language like C++.

It is often impossible for programmers to even know if the function they use has any side effects. For instance, the specification of `std::vector<T>::size() const` does not prevent the implementations from performing side effects, such as logging.

Some side effects are practical to have, and they do not affect the reasoning about predicates in the mathematical sense. These side effects include:

1. Logging, which never affects subsequent computations.
2. Modifying private mutable data members for the purpose of caching function results.
3. Using mathematical functions from `<cmath>`, which store error results in global (thread-local) variable `errno`.
4. Performing scoped locking inside the function, which may affect the execution of other threads.
5. Causing a contract violation handler when runtime-checking the precondition of the function called in the predicate.

Our choice follows the existing practice with `assert()`: it allows side effects, but a lot of advice comes with it, saying that side effects in the predicate cannot be relied upon.

[P0542R5] specified that invoking any side effect inside a contract annotation predicate is undefined behavior. This allowed certain practical optimizations, such as not performing two consecutive evaluations of the same predicate, which is often the case when postcondition of one function is the same as the precondition of another function. It also allowed checking the same predicate twice, for instance once inside the function body, and the second time in the calling context.

While we drop this undefined behavior on any side effects in a contract predicate, we allow similar code transformations, and introduce narrower cases that trigger undefined behavior: in situations where the programmer is clearly doing something nasty.

Why not just follow [P0542R5] and say that any side effect inside a predicate is undefined behavior? Because we do not want programs with "benign" side effects to be penalized, and render additional surprises. Only "malignant" side effects should trigger undefined behavior. We have a pretty good definition of "benign" side effect in a predicate: it does not modify any object that it odr-uses and it does not affect the result of any other subsequent predicate in the same correctness test sequence.

Why not just use simpler wording and say that a predicate listed in a contract annotation can be evaluated an unspecified number of times (including 0 times)? While this would send a clear signal to the users, that the side effects cannot be relied upon, it would not allow the predicate transformations that we described earlier.

7.8. Why include side effect elision in the MVP {rat.eli}

First, it cannot be added later, because then it would be a breaking change. User may start to rely on the mandated side effects in *Check_and_abort* mode, as per the Hyrum's Law, and these would suddenly disappear in future releases.

Second, it gives a strong encouragement to the users not to put side effects in their predicates. Their side effects may disappear, even in *Check_and_abort* mode; or they can be duplicated.

7.9. Why allow access to private and protected members

Programming guidelines often recommend that in contract predicates of public member functions one should only use the public interface of the class. This is in case the class user needs to manually check if the contract is satisfied for an object whose state is not known. However, this is only a guideline, and enforcing it in the language would break other use cases that do not subscribe to the above advice.

In general, the users must *ensure* that the precondition of the called function is satisfied. If they do that, they do not have to check the precondition.

Allowing the access to protected and private members enables a practical usage scheme. In general, function precondition is something that cannot be fully expressed as C++ expression. Implementer choose how much of the function precondition they want to check. They may choose to check some parts of the precondition by accessing private members that they do not want to expose to the users, for instance, because the private implementation may change over time or under configuration:

```
class callback
{
#if !defined NDEBUG
  mutable int _call_count = 0;
#endif

  // ...

public:
  void operator() const
    // real contract: this function can be called no more than 8 times,
    // so the precondition is that the function has been called 7 or less times

#if !defined NDEBUG
    // attempt to check the precondition
    [[pre: _call_count <= 7]];
```

```
#endif
};
```

In the above example, the precondition can only be checked in debugging mode. Once `NDEBUG` is defined, member `_call_count` is removed and there is no way to test the precondition.

Also, a hypothetical constraint to use only public members in contract predicates could result in programmers turning their private and protected members into public members only to be able to express the pre- and postconditions, which does not sound like a good class design.

This has been described in detail in [P1289R1], and in fact adopted by EWG.

7.10. Why not mandate a default translation mode

We recommend that the default mode is *Check_and_abort*, but we do not require this of the implementations. The reason for that is that we believe that it is not possible to mandate this behavior in this International Standard.

Our ideal is that users get safety by default and performance on request. ("Safety" in this context is understood as "do not let the program with a bug execute".) But what the users often use is an IDE. Even if we were able to require of the compiler that its default mode is *Check_and_abort*, the IDE can provide its own defaults which map on non-default values of compiler switches.

This has been discussed in detail in [P1769R0].

7.11. No gratuitous `const` qualification

Ideally, we would like contract predicates to be referentially transparent (have no side effects, depend on nothing else but the objects directly referenced in the expression). This goal is not realistically attainable. We could get closer, however, if we required that all objects referenced in the predicate are treated as if they were `const`. Thus, only operators and functions (including member functions) that take arguments by value or a reference to `const` are allowed.

This could cause some discrepancies in case we have two overloads: one for type `T const&` and the other for `T&` doing different things:

```
struct X
{
  bool is_fine() const { return /* impl 1 */; }
  bool is_fine()       { return /* impl 2 */; }

  friend void fun(X& x)
    [[pre: x.is_fine()]]    // uses impl 1
  {
    [[asset: x.is_fine()]]; // uses impl 2
  }
};
```

Also, there exist referentially transparent functions that do not mark their reference arguments as `const`. While programmers are often advised to mark any functions that do not modify their arguments by contract as `const`, this is just and device, and does not have to be followed. And it is not clear if such functions should be excluded from contract predicates.

Because of these issues, and unclear gains, we do not propose to treat objects as `const` inside contract annotations.

7.12. Interaction with `constexpr` functions

During core constant expression evaluation in *Check_and_abort* mode when any function precondition, postcondition or assertion is violated, the program is ill-formed.

Ideally, we would like such evaluation to be ill-formed even in *Ignore* mode. However, we are not sure if this is easily implementable on all platforms. This proposal allows but not requires this.

7.13. Unimplementable security restrictions {rat.sec}

C++20 had wording "There should be no programmatic way of setting, modifying, or querying the build level of a translation unit." Technically, we could add a similar requirement for the translation mode.

We decided to drop this requirement, as there is no way to actually test if it is satisfied, and it is unrealistic to expect that of the implementations. For instance, A program can open a configuration file of the project that lists the translation mode. Should programs be banned from opening and reading such file?

7.14. No placeholder syntax for "meta-annotations" on contract annotations

It is clear that at some point there will have to be a way to annotate contract annotations with additional information controlling how contract annotations should be interpreted, such as:

1. That a precondition is a new one, and if it is runtime-checked and fails, it should not trigger a call to `std::abort()`.
2. That a precondition is expensive to evaluate, relative to the cost of function body.
3. That the expression in the precondition cannot be odr-used, and therefore evaluated.

We could provide the placeholder syntax for these annotations still in the MVP, so that C++ compilers can start parsing it as soon as possible, before we standardize individual "meta-annotations". This way, programs written in C++30 would compile in C++23 compilers (assuming this paper is accepted for

C++23).

The serious concern about this idea is that while these meta-annotations will be parsed in C++23, they will have different semantics than in the future revisions of C++. This silent change of semantics is unacceptable. For this reason, we do not propose the placeholder syntax for meta-annotations.

We would end up in the situation, where the same program compiles on both older and newer contract support frameworks, but has different semantics. Consider the following program, with possible future extensions:

```
void f(const char* s)
  [[pre{axiom}: is_null_terminated(s)]]; // name must not odr-used

bool binary_search(container<Val>& c, Val const& v)
  [[pre{audit}: is_sorted(c)]];          // must not be evaluated

bool foo(int i)
  [[pre{review}: i >= 0]];               // must not abort

bool bar(int i)
  [[pre{default}: i >= 0]];              // must evaluate and abort
```

No single semantic that doesn't understand the additional syntax can accommodate these expectations.

7.15. Interleaving preconditions and postconditions

This proposal allows to declare preconditions and postconditions in arbitrary order:

```
int f(int x, int y)
  [[post r: p(r)]]
  [[pre: p(x)]]
  [[post r: q(r)]]
  [[pre: p(y)]];
```

We could be stricter about is and require that no precondition annotation can follow any precondition annotation. However:

- There is no ambiguity when these annotations are declared in arbitrary order.
- This looks more like a question of good style, which can be enforced by other tools or coding practices.
- It can sometimes add clarity when pairs precondition-postcondition are grouped together:

```
int f(int x)
  [[pre: neat(x)]]
  [[post r: neat(r)]]

  [[pre: nice(x)]]
  [[post r: nice(r)]];
```

7.16. `abort()` vs `terminate()`

In this proposal, throwing from the predicate calls `std::terminate()` while a failed runtime check calls `std::abort()`. (We simply forbid the implementations to end the violation handler by throwing an exception: it is up to implementations how this requirement is satisfied.)

The above distinction reflects the fundamental difference between the two situations. Throwing from the predicate is a random, unpredictable, but correct situation in the program. Maybe a comparison had to allocate memory, and this allocation failed, because today the server is exceptionally busy. We want to handle it the way we usually handle exceptions when there is no suitable handler: `std::terminate()` is an exception handler, with its unique control flow, however harsh.

In contrast, failing a runtime correctness test is an indication of a bug, and it is not clear if `std::terminate()`, which is the second level of exception handling mechanism, is a suitable tool.

7.17. `[[assert:]]` is not an expression

In this proposal `[[assert:]]` can only appertain to a null statement, which effectively makes it a statement, and it cannot be used as an expression. This is a downside compared to macro `assert()`, which can be used as a subexpression of a bigger expression and therefore be used to protect things like initialization of variables:

```
MyClass::MyClass(int i)
  : member((assert(i > 0), i))
{}
```

In the future `[[assert:]]` could be extended and become an expression. However, this is not in scope of the Minimum Viable Product.

7.18 Contract-based optimizations {rat.cbo}

This proposal does not allow code transformations based on assumption that predicates in correctness annotations always evaluate to `true`. Such transformations could change the program (even parts far before the contract annotation) in a surprising way.

We believe that removing this provision increases the level of consensus.

On the other hand, we believe that the Standard does not have to explicitly allow such transformations. First, they can never be guaranteed or required: a compiler vendor must choose to make an effort and implement them. If a compiler vendor implements such transformations, it can offer the feature as a non-conforming extension, such as `__builtin_unreachable()`.

In fact, this proposal indirectly allows the programmer to achieve the same effect, provided that the compiler vendor provides the necessary parts. This proposal says that what happens in response to contract violation is implementation defined. A conforming implementation may allow the programmer to install a callback to be called on such occasion (a contract violation handler). The programmer may then choose to put `__builtin_unreachable()` inside the handler. The implementation then would have to guarantee that the handler is inlined.

This should not be alarming to people focused on safety, because the above scenario requires the usage of `__builtin_unreachable()`, which is easily detectable through simple mechanical checks, and cannot be installed at runtime.

8. Future compatibility {fut}

While we propose to drop a number of features from the MVP, this proposal does not close the door for adding them in the future, once (if) the MVP has been agreed upon and baked.

To guarantee backward compatibility as we add features in the future, all semantics added in the MVP must not change. Therefore all new features will have to be opt-ins: enabled either through new syntax/annotations, or through new translation modes.

The ability to install a custom violation handler (along with the handler's interface) can be provided as a future extension, with the behavior mandated by the MVP being the semantics of the default violation handler.

The ability to continue after logging the contract violation has two use cases:

1. Apply the "continue" semantics for all contract annotations when running the variant of a program with contract checking enabled for the first time.
2. Applying the "continue" semantics selectively only for the newly added contract annotations.

In the first case, the application of the "continue" semantics is not a default behavior, and would require that a person who assembles the program instructs the compiler to use the special behavior. This can be added in the future as a third mode of translating the source code with contract annotations (in addition to *Ignore* and *Check_and_abort*).

In the second case, there is a need to discriminate the newly added contract annotations from the stable ones. This would require some additional syntax to mark such annotations, for instance:

```
int f(int * p)
  [[pre: p]]            // stable annotation
  [[pre: *p > 0; new]]  // new annotation
;
```

This can be added in the future by introducing a new syntax for the newly added annotations and allowing the programmer to control separately what runtime code should be generated from these "new" annotations.

The syntax space for additional information in contract annotations is quite broad. The alternatives include:

```
  [[post r: r > 0: new]]
  [[post{new} r: r > 0]]
```

Regarding the throwing violation handlers, the only known and well explored use case is for "negative" unit-testing. We note that this ability would only be used in special programs: ones that execute unit tests. It would also require of the tested functions not to be marked as `noexcept`. For this special case it seems reasonable to expect of the person that assembles the program that they instruct the compiler in an explicit way that a unit-test program is built. This ability could be added as a future extension by introducing a yet another translation mode where exceptions thrown from violation handlers are not immediately turned into a call to `std::terminate()` (but they can still be turned into `std::terminate()` when `noexcept` functions are executed in unit-test programs).

It is also possible to do negative testing of preconditions and postconditions by offering a set of reflection features tailored to that eventuality, without even running the function. A strawman proposal could be `std::are_preconditions_satisfied(f, x, y, z) -> bool`.

This might look like a lot of modes of translation, but it should be kept in mind that the perspective of a person that looks at the code will not have modes: a declaration starting with `[[pre:` is always a precondition: something that evaluates to `true` in correct programs. The modes will be visible only to the person assembling the program, and in this case, having a lot of fine grained control is desired.

The point of this section is to illustrate that dropping features like violation handlers, continuation after a failed runtime check or throwing violation handlers from the MVP does not build any technical barriers that would prevent the addition of these features in the future revisions of the contract support framework. We also expect that once the syntax for declaring contract annotations has been standardized, compiler vendors will offer non-standard extensions that will allow the users to experiment with additional features and become a basis for the future standardization.

8.1. Capturing state on function entry {fut.old}

It is sometimes desirable to express in the postcondition how the state of the program upon return from the function is different from the state that the program had when the function was entered. One notable example of this is when we want to check that after a successful `push_back`, the size of the sequence container upon return is greater by one than the size of the container upon function entry. However, by the time the precondition is evaluated, the previous state of the container is no more. We would have to make a *copy* of the portions of the program state that would be later inspected in the postcondition.

There could be at least two ways to express this. One is to introduce even more names in the postcondition declaration, and use a syntax similar to the one used in lambda-captures:

```
int f(int& i, array<int, 8>& arr)
  [[post r, old_i = i: r >= old_i]]        // value of i upon entry
  [[post r, old_7 = arr[7]: r >= old_7]];  // value of arr[7] upon entry
```

The other option is to use a new keyword (or token) for this purpose, like `oldof` proposed in [N1962]:

```
int f(int& i, array<int, 8>& arr)
  [[post r: r >= oldof(i)]]        // value of i upon entry
  [[post r: r >= oldof(arr[7])]];  // value of arr[7] upon entry
```

This future addition would also address the case where a postcondition refers to the non-reference function parameter (see section {rat.arg}), but the function cannot afford to add `const`:

```
template<class ForwardIt, class T>
ForwardIt find(ForwardIt first, ForwardIt last, const T& value)
  [[post r: distance(oldof(first), r) >= 0u]]  // `first` not referenced
  [[post r: distance(r, last) >= 0u]]
{
  for (; first != last; ++first) {
    if (*first == value) {
      return first;
    }
  }
  return last;
}
```

This proposal is open for either of these syntax variants (in the future).

9. Open issues {ope}

9.1. Syntax for introducing names in postconditions {ope.syn}

It has been suggested that postconditions could have a better syntax for introducing the name of the returned value. Any such exploration of syntactic space should take into account that postconditions might need to introduce more names, as described in section {fut.old}.

```
int f(int& i, array<int, 8>& arr)
  [[post r, old_i = i: r >= old_i]]        // value of i upon entry
  [[post r, old_7 = arr[7]: r >= old_7]];  // value of arr[7] upon entry
```

An alternate syntax for postconditions that does not preclude the option above in the future would be to use parentheses:

```
int f(int& i, array<int, 8>& arr)
  [[post(r): r >= 0]]                       // for the MVP
  [[post(r, old_i = i): r >= old_i]]        // future
  [[post(r, old_7 = arr[7]): r >= old_7]];  // future
```

7. Wording {wor}

Wording changes are against N4892.

The text in bluish font indicates comments that are intended to easily map the wording onto the overview part of the proposal, and to give a rationale for the wording choices. We use the following terms:

- *contract annotation* — either a precondition or a postcondition.
- *correctness annotation* — either an assertion or a contract annotation.
- *correctness test* — is an "event" corresponding to a correctness annotation — it may be a no-op in *Ignore* mode or produce instructions in *Check_and_abort* — which can be sequenced before or after other instructions.

In 6.3/13 [basic.def.odr], add a new bullet after bullet 13.13:

> if `D` invokes a function with a contract annotation, or is a function that contains an assertion or has a contract annotation (9.12.4.2), it is implementation-defined under which conditions all definitions of `D` shall be translated using the same translation mode (9.12.4.3); and

The above permits the implementations to allow the co-existence of two definitions of the same function compiled in two different translation modes. The above takes into account that postconditions, much like preconditions, can also be evaluated outside of their function.

Modify the beginning of 6.9.1/11 [intro.execution] as follows:

> When invoking a function (whether or not the function is inline), every argument expression and the postfix expression designating the called function are sequenced before the function's precondition test sequence (9.12.4.3); the function's precondition test sequence is sequenced before every expression or statement in the body of the called function.

In 11.4.1/7 [class.mem.general], add a new bullet after bullet 7.4:

— contract annotation (9.12.4.2), or

In 7.6.1.3 [expr.call], add a new paragraph after paragraph 7:

The function's precondition test sequence (9.12.4.3) is sequenced after the initialization of function parameters. The function's postcondition test sequence (9.12.4.3) is sequenced before the destruction of function parameters.

In 7.7/5 [expr.const], add a new bullet after bullet 5.12:

— an unsuccessful correctness test (9.12.4.3), or

In 9.12.1/1 [dcl.attr.grammar], modify the production for *attribute-specifier* as follows:

…
attribute-specifier:

- `[ [` *attribute-using-prefix*$_{opt}$ *attribute-list* `] ]`
- *correctness-specifier*
- *alignment-specifier*

…

Contract annotations are not regular attributes, but we still define them in [dcl.attr] because (1) they share some characteristics of attributes, and (2) we follow the precedent of `alignas`.

Add a new section 9.12.4 Correctness specifiers [dcl.correct] ([dcl.attr.deprecated] becomes 9.12.5).

Add a new section 9.12.4.1 Syntax [dcl.correct.syn]:

1. Correctness specifiers are used to specify preconditions, postconditions, and assertions for functions.

correctness-specifier:

- `[ [ pre :` *conditional-expression* `] ]`
- `[ [ post` *identifier*$_{opt}$ `:` *conditional-expression* `] ]`
- `[ [ assert :` *conditional-expression* `] ]`

2. A *correctness-specifier* using `pre` is a *precondition*. The specifier may be applied to the function type of a function declaration. [*Note*: A precondition expresses a predicate, which can be evaluated upon entry into the function. If evaluated and the result is `false`, this indicates that the program contains a bug. — *end note*].

Term "bug" seems fine inside a non-normative note. This formulation in the note looks more unambiguous than an alternative that would say "function's expectation".

3. A *correctness-specifier* using `post` is a *postcondition*. The specifier may be applied to the function type of a function declaration. [*Note*: A postcondition expresses a predicate which can be evaluated when the function returns to its caller. If evaluated and the result is `false`, this indicates that the program contains a bug. — *end note*] A postcondition may introduce an identifier to represent the glvalue result or the prvalue result object of the function. When the declared return type of a non-templated function contains a placeholder type, the optional *identifier* shall be present only in a definition. [*Example*:

```
int f(int& p)
  [[post: p >= 0]]     // OK
  [[post r: r >= 0]];  // OK

auto g(auto& p)
  [[post: p >= 0]]     // OK
  [[post r: r >= 0]];  // OK

auto h(int& p)
  [[post: p >= 0]]     // OK
  [[post r: r >= 0]];  // error: cannot name the return value

auto h(int& p)
  [[post: p >= 0]]     // OK
  [[post r: r >= 0]]   // OK
{
  return p = 0;
}
```

— *end example*]

4. A *correctness-specifier* using `assert` is an *assertion*. The specifier may be applied to a null statement (8.3). An assertion correctness test (9.12.4.3) is performed as part of the evaluation of the null statement the assertion applies to. [*Note*: An assertion expresses a predicate. If evaluated and the result is `false`, this indicates that the program contains a bug. — *end note*]

5. Preconditions, postconditions, and assertions are collectively called *correctness annotations*. The *conditional-expression* in a correctness annotation is contextually converted to `bool` (7.3); the converted expression is called the *predicate* of the correctness annotation. [*Note*: The

predicate of a correctness annotation is potentially evaluated (6.3). — *end note*]

Add a new section 9.12.4.2 Contract annotations [dcl.correct.anno]:

1. A *contract annotation* is a precondition or a postcondition. A contract annotation may be applied to the function type of a function declaration. The first declaration of a function *D* shall specify all contract annotations (if any) of the function. Other declarations of the function reachable (10.7) from *D* shall either specify no contract annotations or the same list of contract annotations; no diagnostic is required if corresponding conditions will always evaluate to the same value. If declarations of function *F* appear in two translation units, the first declarations of *F* in either translation unit their lists of contract annotations shall be the same; no diagnostic required; also no diagnostic is required if corresponding conditions will always evaluate to the same value. If a friend declaration *D* is the first declaration of the function in a translation unit and has a contract annotation, that declaration shall be a definition and there shall be no other declaration of the function or function template which is reachable from *D* or from which *D* is reachable.

The above means that if a friend function declaration introduces a new function and has contract annotations, it has to be a "hidden friend". This requirement was copied from [P0542R5].

2. Two lists of contract annotations are the same if they consist of the same contract annotations in the same order. Two contract annotations are the same if their predicates are the same. Two predicates contained in *correctness-specifier*s are the same if they would satisfy the one-definition rule (6.3 **`[basic.def.odr]`**) were they to appear in function definitions, except for renaming of parameters, return value identifiers (if any), and renaming of template parameters.

3. [*Note*: A function pointer cannot include contract conditions. [*Example*:

```
typedef int (*pfa)() [[post r: r != 0]]; // error: contract annotation not on a function declaration

int g(int x)
  [[pre: x != 0]]
  [[post r: r != 0]];

int (*pf)(int) = g; // OK
int x = pf(5);      // contract annotations of g are tested
```

— *end example*] — *end note*]

4. The predicate of a contract condition has the same semantic restrictions as if it appeared in the *noexcept-specification* of the function it applies to, except that the return type of the function is known in a contract condition appertaining to its definition, even if the return type contains a placeholder type. [*Example*:

```
class X {
private:
  int m;
public:
  void f() [[pre: m > 0]];             // OK
  friend void g(X x) [[pre: x.m > 0]]; // OK
};

void h(X x) [[pre: x.m > 0]];          // error: m is a private member
```

— *end example*]

5. A precondition test (9.12.4.3) is performed immediately before starting evaluation of the function body. [*Note*: The function body includes the *function-try-block* (Clause 14) and the *ctor-initializer* (11.9.3). — *end note*] A postcondition test is performed immediately before returning control to the caller of the function. [*Note*: The lifetime of local variables and temporaries has ended. Exiting via an exception or via `longjmp` (17.13.3) is not considered returning control to the caller of the function. — *end note*]

6. The *basic precondition test sequence* of a given function `f` is the sequence of precondition tests corresponding to the list of preconditions of `f`. For two precondition annotations *P1* and *P2* applied to a function, if *P1* appears lexically before *P2* then the precondition test corresponding to *P1* is sequenced before the precondition test corresponding to *P2*. If a function has multiple postconditions, their tests (if any) will be performed in the order they appear lexically. The *basic postcondition test sequence* of a given function is the sequence of postcondition tests corresponding to the list of postconditions of the function. For two postcondition annotations *P1* and *P2* applied to a function, if *P1* appears lexically before *P2* then the postcondition test corresponding to *P1* is sequenced before the postcondition test corresponding to *P2*. [*Note*: A basic precondition test sequence can be empty if the function has not preconditions. — *end note*] [*Example*:

```
void f(int* p, int*& q)
  [[post: q != nullptr]] // #4
  [[pre:  p != nullptr]] // #1
  [[post: *q >= 0]]      // #5
  [[pre:  *p >= 0]]      // #2
{
  q = p;                 // #3
}
```

The numbers indicate the order in which expressions in preconditions, postconditions and function body are evaluated. The basic precondition test sequence consists of two precondition tests corresponding to #1 and #2. The basic postcondition test sequence consists of two postconditions corresponding to #4 and #5. — *end example*]

7. If a predicate in the postcondition odr-uses (6.3) a non-reference parameter, this parameter shall be defined `const` inside the function definition. [*Example*:

```
int f(int i)
  [[post r: r == i]];

int g(int i)
  [[post r: r == i]];

int f(const int i)  // OK
{
  return i;
}

int g(int i)        // error: i is not declared const
{
  return i;
}
```

— *end example*]

8. The postcondition test sequence (9.12.4.3) is sequenced after the initialization of the returned value, and after the destruction of objects with automatic storage duration declared in the function body. The postcondition test sequence is sequenced before the destruction of function arguments. [*Note*: Therefore, a postcondition can inspect the state of the return value and function arguments. — *end note*]

Add a new section 9.12.4.3 Testing contract annotations [dcl.correct.test]:

1. A translation may be performed in one of the following *translation modes*: *ignore*, or *enforce*. The mechanism for selecting the translation mode is implementation-defined. The translation of a program consisting of translation units where the translation mode is not the same in all translation units is conditionally-supported with implementation-defined semantics.

2. *Recommended practice*: If no translation mode is explicitly selected, *enforce* should be the default translation mode.

C++20 had wording "There should be no programmatic way of setting, modifying, or querying the build level of a translation unit." We omit is as explained in section {rat.sec}.

3. An *ignored correctness annotation test* performs no operation. [*Note*: The predicate is potentially-evaluated (6.3). —*end note*]

4. An *enforced correctness annotation test* is performed as follows. The predicate is evaluated. If the evaluation exits via an exception, `std::terminate()` is called. If the evaluation exits via a call to `longjmp` (17.13.3) the behavior is undefined. An implementation is allowed to substitute the evaluation of a predicate `P` that returns to the caller with an alternative predicate that returns the same value as `P` but has no side effects. An enforced correctness annotation test where the evaluation of `P` returns `false` is called *unsuccessful*. For an unsuccessful correctness annotation test the *contract violation handler* is invoked. The contract violation handler does not exit via an exception or via a call to `longjmp` (17.13.3). The semantics of the contract violation handler are implementation defined.

5. *Recommended practice*: The contract violation handler should by default output to the stream buffer associated with the object `stderr`, declared in `<cstdio>` (29.13.1), the message containing the textual representation of the predicate and the location in the source code where the predicate was evaluated. For preconditions the source location should be that where the function containing the corresponding contract annotation was called. For postconditions, the source location should be inside the function definition.

6. [*Note*: The contract violation handler does not have to be a function with any linkage. An implementation may provide a way to customize the behavior of the contract violation handler. — *end note*]

However, an implementation might want to implement it as a global pointer to function, lest when in the future the Standard starts to require an installable violation handler, this should cause an ABI breakage.

7. After the contract violation handler is executed, the program exits and an implementation-defined form of the status *unsuccessful termination* is returned. No destructors for objects of automatic, thread, or static storage duration are executed. Functions passed to `atexit()` (6.9.3.4) are not called.

8. In translation mode *enforce* all correctness annotation tests are enforced. In translation mode *Ignore* during constant evaluation (7.7) it is implementation-defined whether correctness annotation tests are ignored or enforced. In all other situations in *Ignore* translation mode all correctness annotation tests are ignored.

9. If a correctness annotation test modifies the value of any object odr-used (6.3) in its predicate, the behavior is undefined.

10. Let `PRE` denote the basic precondition test sequence of a given function `f`. When performing enforced contract annotation tests, it is unspecified if `PRE` is performed one or two times. This is called a *precondition test sequence*. Let `POST` denote the basic postcondition test sequence of a given function `f`. When performing enforced contract annotation tests, it is unspecified if `POST` is performed one or two times. This is called a *postcondition test sequence*. [*Note*: This allows the implementations to test the precondition and postcondition of a function both inside and outside of the function body. — *end note*]

11. When the results of functions `fs` are used to initialize function arguments of a function `g`, the *combined test sequence* is a sequence of postcondition test sequences of `fs` followed by a precondition test sequence of `g`. [*Example*:

```
int f1() [[post r: p1(r)]];
int f2() [[post r: p2(r)]];
```

```
void g(int i1, int i2) [[pre: p1(i1)]] [[pre: p2(i2)]];

g(f1(), f2());
```

The combined test sequence of this function call graph is `p1(r)`, `p2(r)`, `p1(i1)`, `p2(i2)`. — *end example*] If two expressions `E1` and `E2` appearing in a combined test sequence are same (9.12.4.2) and `E1` is sequenced before `E2` the implementation is allowed to replace the evaluation of `E2` with the result of `E1`. If such substitution changes the result of the test containing `E1` and te test is performed, the behavior is undefined. [*Example*:

```
int f1() [[post r: p1(r)]];
int f2() [[post r: p2(r)]];
void g(int i1, int i2) [[pre: pg()]] [[pre: p1(i1) && p2(i2)]];

g(f1(), f2());
```

The implementation is allowed to skip the evaluation of `p1(i1)` and `p2(i2)` and instead use the values previously returned by `p1(r)` and `p2(r)`. — *end example*] [*Example*:

```
int f1() [[post: q()]];
int f2() [[post: q()]];
void g(int i1, int i2);

g(f1(), f2());
```

The implementation is not allowed to substitute any evaluation of `q()` with the result of the other, because their corresponding tests are indeterminately sequenced. — *end example*]

In 11.7.3 [class.virtual] add a new paragraph after 11.7.3/17:

18. If an overriding function specifies contract annotations (9.12.4), it shall specify the same list of contract annotations as its overridden functions; no diagnostic is required if predicates of corresponding contract annotations always evaluate to the same value. Otherwise, it is considered to have the list of contract annotations from one of its overridden functions; the names in the contract annotations are bound, and the semantic constraints are checked, at the point where the contract annotations appear. Given a virtual function `f` with a contract annotation that odr-uses `*this` (6.3), the class of which `f` is a direct member shall be an unambiguous and accessible base class of any class in which `f` is overridden. If a function overrides more than one function, all of the overridden functions shall have the same list of contract annotations (9.12.4); no diagnostic is required if predicates in corresponding annotations will always evaluate to the same value. [*Example*:

```
struct A {
  virtual void g() [[pre: x == 0]];
  int x = 42;
};

int x = 42;
struct B {
  virtual void g() [[pre: x == 0]];
}

struct C : A, B {
  virtual void g(); // error: precondition annotations of overridden functions are not the same
};
```

— *end example*]

In 13.8.3.3/3 [temp.dep.expr] add a bullet after bullet 3.6:

— an *identifier* introduced in a postcondition ([dcl.correct]) to represent the result of a templated function whose declared return type contains a placeholder type,

In 13.9.4 [temp.expl.spec] add a note after 13.9.4/13:

14. [*Note*: For an explicit specialization of a function template, the contract annotations (9.12.4) of the explicit specialization are independent of those of the primary template.— *end note*]

In 14.5.1/1 [except.terminate] add a new bullet after bullet 1.7:

— when a correctness annotation test (9.12.4) exits via an exception, or

8. Acknowledgments {ack}

Daveed Vandevoorde offered useful feedback on the syntax of contract annotations.

Joshua Berne reviewed this document, including the wording, and offered many useful suggestions.

Walter Brown reviewed the document, and offered many a correction, clarification and improvement.

Nathan Myers offered insights and guidance on the tricky corner cases of contract annotations.

Jens Maurer reviewed the proposed wording and offered significant improvements.

Ville Voutilainen offered numerous useful suggestions to improve the quality of the paper.

John McFarlane offered useful suggestions that improved the quality of the paper.

9. References {ref}

- [N1613] — Thorsten Ottosen, "Proposal to add Design by Contract to C++" (http://www.open-std.org/JTC1/SC22/WG21/docs/papers/2004/n1613.pdf).
- [N1800] — Lawrence Crowl, Thorsten Ottosen, "Contract Programming For C++0x" (http://www.open-std.org/JTC1/SC22/WG21/docs/papers/2005/n1800.pdf).
- [N1962] — Lawrence Crowl, Thorsten Ottosen, "Proposal to add Contract Programming to C++ (revision 4)" (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2006/n1962.html).
- [N4110] — J. Daniel Garcia, "Exploring the design space of contract specifications for C++" (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4110.pdf).
- [N4075] — John Lakos, Alexei Zakharov, Alexander Beels, "Centralized Defensive-Programming Support for Narrow Contracts(Revision 6)" (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4075.pdf).
- [N4135] — John Lakos, Alexei Zakharov, Alexander Beels, Nathan Myers, "Language Support for Runtime Contract Validation (Revision 8)" (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4135.pdf).
- [P0380R0] — G. Dos Reis, J. D. Garcia, J. Lakos, A. Meredith, N. Myers, B. Stroustrup, "A Contract Design" (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0380r0.pdf).
- [P1494R1] — S. Davis Herring, "Partial program correctness" (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1494r1.html).
- [P2176r0] — Andrzej Krzemieński, "A different take on inexpressible conditions" (https://isocpp.org/files/papers/P2176R0.html).
- [P2339R0] — Andrzej Krzemieński, "Contract violation handlers" (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2339r0.html).
- [P2358R0] — Gašper Ažman, John McFarlane, Bronek Kozicki, "Defining Contracts" (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2358r0.pdf).
- [P0542R5] — G. Dos Reis, J. D. Garcia, J. Lakos, A. Meredith, N. Myers, B. Stroustrup, "Support for contract based programming in C++" (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0542r5.html).
- [P1289R1] — J. Daniel Garcia, Ville Voutilainen, "Access control in contract conditions" (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1289r1.pdf).
- [P1323R2] — Hubert S.K. Tong, "Contract postconditions and return type deduction" (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1323r2.html).
- [P1606R0] — Joshua Berne, "Requirements for Contract Roles" (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1606r0.pdf).
- [P1769R0] — Ville Voutilainen, Richard Smith, "The «default» contract build-level and continuation-mode should be implementation-defined" (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1769r0.html).
- [P2038R0] — Andrzej Krzemieński, Ryan McDougall, "Proposed nomenclature for contract-related proposals" (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2038r0.html).
- [P2182R1] — Andrzej Krzemieński, Joshua Berne, Ryan McDougall, "Contract Support: Defining the Minimum Viable Feature Set" (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2182r1.html).
- [P2036R1] — Barry Revzin, "Change scope of lambda *trailing-return-type*" (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2036r1.html).

Appendix A. Open issues with continuing violation handlers {apa}

We are aware of two unintuitive consequences of continuing violation handlers. First, in case the precondition is guarding against the abstract machine violation (hard UB), logging and then reaching the point of the abstract machine violation might result in what would be observed as removing the log entry for the contract violation. This has been described in detail in [P2339R0] as well as in [P1494R1]. This is no worse than disabling runtime checking altogether (which is uncontroversial), but can really fool whoever troubleshoots the bug: we see no contract-violation log entry, so we think control never reached this point, even though it did. Thus, the continuing violation handler has the potential to deceive whoever uses contract violation logs. There is no agreement on how realistic the possibility of encountering this effect is. The solution presented in [P1494R1] has the potential to address the above issue. But until this is explored, any wider contract proposal that allows continuation would be blocked on [P1494R1].

Second, the continuation mode can introduce new abstract machine violations. Going back to the example provided earlier:

```
int f(int * p)
  [[pre: p]]
  [[pre: *p > 0]]
{
  if (!p)             // safety double-check
    throw Bug{};

  if (*p <= 0)        // safety double-check
    throw Bug{};

  return *p - 1;
}
```

If this function is invoked in a program that doesn't runtime-check contract annotations, it behaves in a tolerable way for `p == nullptr`: it throws an exception. But when runtime checking is enabled and the program is allowed to continue after the failed check, this code is equivalent to:

```
int f(int * p)
{
  if (!p)             // (1)
    log_violation();

  if (*p <= 0)        // (2)
    log_violation();

  if (!p)             // safety double-check
    throw Bug{};

  if (*p <= 0)        // safety double-check
    throw Bug{};

  return *p - 1;
}
```

If `p` happens to be null, check (1) will fail and the violation will get logged. The program will proceed to check (2) and there, it will dereference the null pointer causing an Abstract Machine Violation (hard UB). The key observation here is that the defensive checks that throw exceptions (or return) have the "short circuiting" property: if one fails, the subsequent ones are not executed:

```
int f(int * p)
{
  if (!p)
    throw Bug{};

  if (*p <= 0)        // null `p` never dereferenced
    throw Bug{};

  return *p - 1;
}
```

Short-circuiting is also guaranteed for subsequent precondition annotations, provided that the contract violation logging ends in calling `std::abort()`. But short-circuiting is gone, when the handler allows the program flow to continue.

Splitting a precondition into smaller chunks is used for at least two purposes:

1. Obtaining as fine-grained information as possible from contract violation logs.
2. Differentiating cheap and expensive checks, for the purpose of controlling their behavior separately.

Until this problem is addressed, the continuation after a runtime-evaluated check is a potentially dangerous feature that can introduce an Abstract Machine Violation (hard UB) on top of a BizBug (a programmer bug). While this problem may be solvable, the analysis and solution will take time, which will delay the adoption of the minimum contract support if it allows the continuation.